

Report Lab10

ตอบคำถาม

1.Reactive Programming vs Blocking ต่างกันอย่างไร

Blocking Thread จะรอจนกว่างานจะเสร็จ แต่ reactive จะทำตัวอื่นต่อ ไม่รอตัวเดิม ทำให้เหมาะกับระบบที่ต้องการความเร็วสูงและเป็นแบบ streaming

2. Mono vs Flux ใช้กรณีไหน และทำไม

Mono ใช้กรณีที่ต้อง return 0 หรือ 1 ค่า เพราะ Mono return ได้ค่าเดียว

Flux ใช้กรณีที่ต้อง return 0 ถึง หลายค่า เพราะ Flux สามารถ return ได้หลายค่า การเลือกชนิดของ publisher ควรสื่อความหมายถึงจำนวนข้อมูลที่จะได้รับ

3.WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

.get() ใช้กำหนด HTTP Method ว่าต้องการส่ง GET Request

.uri() ใช้กำหนด URL หรือ path ของ endpoint ที่ต้องการเรียก

.retrieve() ใช้รับ HTTP Response จาก Server

.bodyToMono() ใช้แปลง Response Body ให้เป็น Mono<T>

.bodyToFlux() ใช้แปลง Response Body ให้เป็น Flux<T>

4.อธิบาย code

Repository

Java

```
private final Map<String, Product> store = new ConcurrentHashMap<>();

// — Constructor: ใส่ข้อมูลตัวอย่าง —————
public ProductRepository() {
    store.put("1", new Product("1", "iPhone 15 Pro (673380262-4 SEC 2)",
        "Electronics", "Apple", 50, 39900.0, "MEMBER"));
    store.put("2", new Product("2", "MacBook Air M3",
        "Electronics", "Apple", 20, 49900.0, "NONE"));
    store.put("3", new Product("3", "Samsung Galaxy S24",
        "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
}

// — 1. หา Product 1 รายการ —————
/**
 * TODO: คืน Mono<Product> จาก store โดยใช้ id
 *      ถ้าไม่พบให้คืน Mono.empty()
 *
 * Hint: store.get(id) คืน Product หรือ null
 *      ถ้า null ให้ใช้ Mono.empty()
 *      ถ้ามีค่าให้ใช้ Mono.just(product)
 */
```

```

    */
    // ค้นหา Product จาก id
    // ถ้าพบจะคืน Mono<Product>
    // ถ้าไม่พบจะคืน Mono.empty()
    public Mono<Product> findById(String id) {
        // TODO: เติม code ตรงนี้
        Product product = store.get(id);
        if (product != null) {
            return Mono.just(product);
        } else {
            return Mono.empty();
        }
    }

    // — 2. หา Product ทั้งหมด —————
    /**
     * TODO: คืน Flux<Product> ของทุกรายการใน store
     *
     * Hint: store.values() คืน Collection<Product>
     *       ใช้ Flux.fromIterable(...) แปลงเป็น Flux
     */
    // ดึง Product ทั้งหมด
    // ใช้ Flux เพราะข้อมูลมีได้หลายรายการ
    public Flux<Product> findAll() {
        // TODO: เติม code ตรงนี้

        return Flux.fromIterable(store.values()); // ← แก้งบรทัดนี้
    }

    // — 3. บันทึก Product —————
    /**
     * TODO: บันทึก product ลง store แล้วคืน Mono<Product>
     *
     * Hint: store.put(product.getId(), product)
     *       แล้วใช้ Mono.just(product) คืนค่า
     */
    // บันทึก Product ลงใน store
    // แล้วคืน Product ที่บันทึกกลับมา
    public Mono<Product> save(Product product) {
        // TODO: เติม code ตรงนี้
        store.put(product.getId(), product);
        return Mono.just(product);
    }
}

```

```
// — 4. ลบ Product —————
/**
 * TODO: ลบ product จาก store แล้วคืน Mono<Void>
 *
 * Hint: store.remove(id)
 *        แล้วใช้ Mono.empty() คืนค่า (Mono<Void>)
 */
// ลบ Product จาก id
// ไม่มีข้อมูลที่ต้องคืนกลับจึงใช้ Mono<Void>
public Mono<Void> deleteById(String id) {
    // TODO: เติม code ตรงนี้
    store.remove(id);
    return Mono.empty();
}

// — 5. กรองตาม category —————
/**
 * TODO: คืน Flux<Product> ที่ category ตรงกัน
 *
 * Hint: findAll()
 *        .filter(p -> p.getCategory().equalsIgnoreCase(category))
 */
// กรอง Product ตาม category
// ใช้ filter เพื่อเลือกเฉพาะ Product ที่ category ตรงกัน
public Flux<Product> findByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));
}
```

Service

```
Java
private final ProductRepository repository;

public ProductService(ProductRepository repository) {
    this.repository = repository;
}

// — 1. ดึง Product 1 รายการ —————
```

```

/**
 * TODO: เรียก repository.findById(id) แล้วคืนผล
 *      ถ้าไม่พบให้ throw RuntimeException("Product not found: " + id)
 *
 * Hint: repository.findById(id)
 *      .switchIfEmpty(Mono.error(new RuntimeException(...)))
 */
// ค้นหา Product จาก id
// ถ้าไม่พบข้อมูลจะสร้าง RuntimeException
public Mono<Product> getById(String id) {
    // TODO: เติม code ตรงนี้
    return repository.findById(id)
        .switchIfEmpty(Mono.error(new RuntimeException("Product not found:
" + id)));
}

// — 2. ดึง Product ทั้งหมด —————
/**
 * TODO: เรียก repository.findAll() แล้วคืนผล
 */
// ดึง Product ทั้งหมดจาก Repository
public Flux<Product> getAll() {
    // TODO: เติม code ตรงนี้
    return repository.findAll();
}

// — 3. บันทึก Product —————
/**
 * TODO: เรียก repository.save(product) แล้วคืนผล
 *
 * เพิ่มเติม: ถ้า product.getId() เป็น null ให้ generate id ใหม่
 * Hint: java.util.UUID.randomUUID().toString()
 */
// บันทึก Product
// ถ้า Product ยังไม่มี id จะสร้าง UUID ใหม่ให้อัตโนมัติ
public Mono<Product> save(Product product) {
    // TODO: เติม code ตรงนี้
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }

    return repository.save(product);
}

```

```
// — 4. ลบ Product —————
/**
 * TODO: เรียก repository.deleteById(id) แล้วคืนผล
 */
public Mono<Void> delete(String id) {
    // TODO: เติม code ตรงนี้
    return repository.deleteById(id);
}

// — 5. กรองตาม category —————
/**
 * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
 */
public Flux<Product> getByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return repository.findByCategory(category);
}

// — 6. คำนวณราคาลงส่วนลด —————
/**
 * TODO: หา Product จาก id แล้วคืน discountedPrice
 *
 * Hint: getById(id)
 *       .map(p -> p.getDiscountedPrice())
 */
// ดึงราคาลงส่วนลด
// ใช้ map เพื่อเปลี่ยนจาก Mono<Product> เป็น Mono<Double>
public Mono<Double> getDiscountedPrice(String id) {
    // TODO: เติม code ตรงนี้
    return getById(id).map(p -> p.getDiscountedPrice());
}
```

Controller

```
Java
private final ProductService service;

public ProductController(ProductService service) {
    this.service = service;
}
```

```
// =====  
// ✅ ตัวอย่างที่ทำเสร็จแล้ว - ศึกษาแล้วทำ endpoint ที่เหลือ  
// =====  
  
/**  
 * GET /products/{id}  
 * คืน Mono<Product> - ค้นหา Product 1 รายการ  
 *  
 * ทดสอบ: GET http://localhost:8080/products/1  
 */  
@GetMapping("/{id}")  
public Mono<Product> getById(@PathVariable String id) {  
    return service.getById(id);  
}  
  
// =====  
// ❌ TODO: เติม method body ด้านล่างนี้  
// =====  
  
/**  
 * GET /products  
 * TODO: คืน Flux<Product> ทุกรายการ  
 *  
 * Hint: เรียก service.getAll()  
 * ทดสอบ: GET http://localhost:8080/products  
 */  
// ดึง Product ทั้งหมด  
@GetMapping  
public Flux<Product> getAll() {  
    // TODO: เติม code ตรงนี้  
    return service.getAll(); // ← แก่บรรทัดนี้  
}  
  
/**  
 * POST /products  
 * TODO: รับ Product จาก request body แล้วบันทึก  
 *  
 * Hint: เรียก service.save(product)  
 * ทดสอบ: POST http://localhost:8080/products  
 *      Body: { "name": "...", "price": 999.0, ... }  
 */  
// รับ Product จาก Request Body แล้วบันทึก
```

```
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    // TODO: เติม code ตรงนี้
    return service.save(product); // ← แกักรรทัดนี้
}

/**
 * DELETE /products/{id}
 * TODO: ลบ Product และคืน Mono<Void>
 *
 * Hint: เรียก service.delete(id)
 * ทดสอบ: DELETE http://localhost:8080/products/1
 */
// ลบ Product ตาม id
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    return service.delete(id); // ← แกักรรทัดนี้
}

/**
 * GET /products/category/{category}
 * TODO: คืน Flux<Product> ที่กรองตาม category
 *
 * Hint: เรียก service.getByCategory(category)
 * ทดสอบ: GET http://localhost:8080/products/category/Electronics
 */
// ดึง Product ตาม category
@GetMapping("/{category}/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    // TODO: เติม code ตรงนี้
    return service.getByCategory(category); // ← แกักรรทัดนี้
}

/**
 * GET /products/{id}/price
 * TODO: คืน Mono<Double> ราคาหลังส่วนลด
 *
 * Hint: เรียก service.getDiscountedPrice(id)
 * ทดสอบ: GET http://localhost:8080/products/1/price
 */
// ดึงราคาหลังส่วนลดของ Product
@GetMapping("/{id}/price")
```

```
public Mono<Double> getDiscountedPrice(@PathVariable String id) {  
    // TODO: เติม code ตรงนี้  
    return service.getDiscountedPrice(id); // ← แก่บรรทัดนี้  
}
```

Endpoints

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/products`. The response is a 200 OK status with a JSON array of product data. The response body is displayed in a table format.

id	name	category	brand	stock	price	discountType	discountedPrice	
0	1	iPhone 15 Pro (673380262-4 SEC 2)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

กรรมภัฏ พิริยะ 673380262-4 sec.2

lab10Principle / New Request

GET

http://localhost:8080/products/1

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (2) Test Results 200 OK 25 ms 243 B Save Response

JSON

Preview

Visualize

id	1
name	iPhone 15 Pro (673380262-4 SEC 2)
category	Electronics
brand	Apple
stock	50
price	39900
discountType	MEMBER
discountedPrice	35910

lab10Principle / New Request

POST

http://localhost:8080/products

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

Body form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "Kompat Piriya 673380262-4 sec.2",
3   "category": "Electronics",
4   "brand": "Apple",
5   "stock": 10,
6   "price": 25000.0,
7   "discountType": "MEMBER"
8 }
```

Body Cookies Headers (2) Test Results 200 OK 53 ms 276 B Save Response

JSON

Preview

Visualize

id	4f125aff-06fc-480c-ac7e-79fc10545b5c
name	Kompat Piriya 673380262-4 sec.2
category	Electronics
brand	Apple
stock	10
price	25000
discountType	MEMBER
discountedPrice	22500

กรรมภัฏ พิริยะ 673380262-4 sec.2

lab10Principle / New Request

GET

http://localhost:8080/products/category/Electronics

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description		Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (2)

Test Results

200 OK

11 ms

769 B

Save Response

{}

JSON

Preview

Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380262-4 SEC 2)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920
3	4f125aff-06fc-480c-ac7e-79fc10545b5c	Kompat Piriya 673380262-4 sec.2	Electronics	Apple	10	25000	MEMBER	22500

lab10Principle / New Request

GET

http://localhost:8080/products/1/price

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description		Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (2)

Test Results

200 OK

9 ms

77 B

Save Response

{}

JSON

Preview

Visualize

135910.0

lab10Principle / New Request

SaveShare

DELETE

http://localhost:8080/products/2

Send

DocsParamsAuthorizationHeaders (6)BodyScriptsSettingsCookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

BodyCookiesHeaders (1)Test Results

200 OK · 7 ms · 38 B · Save Response

RawPreviewVisualize

1

lab10Principle / New Request

SaveShare

GET

http://localhost:8080/products/2

Send

DocsParamsAuthorizationHeaders (6)BodyScriptsSettingsCookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

BodyCookiesHeaders (2)Test Results

500 Internal Server Error · 42 ms · 9.67 KB · Save Response

{ } JSON

PreviewDebug with AI

timestamp	2026-09-22T12:38:22.413+00:00
path	/products/2
status	500
error	Internal Server Error
requestId	51ec14c9-7
message	Product not found: 2